一、了解 Web 框架

二、教学目标

1、理解什么是WEB 程序

2、理解网络程序交互三要素的作用

3、掌握 本地 搭建WEB 服务的

4、了解 Django 全栈框架的作用

5、深入理解 MVC\MVT 架构的差异，能对比分析主流框架的设计思想

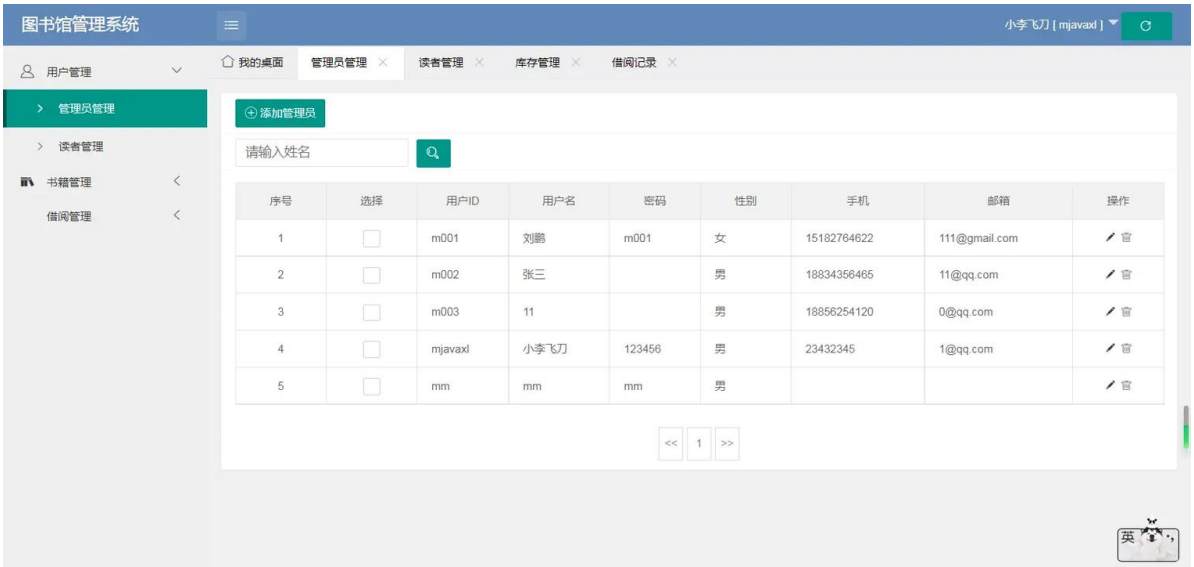
6、能够使用 AI 工具完成基本的辅助开发操作

三、WEB 开发介绍

WEB: 网页程序，基于浏览器访问的程序，可展现不同数据的动态页面

PythonWeb : 用 *Python* 语言技术来解决相关**WEB**互联网领域的技术栈。

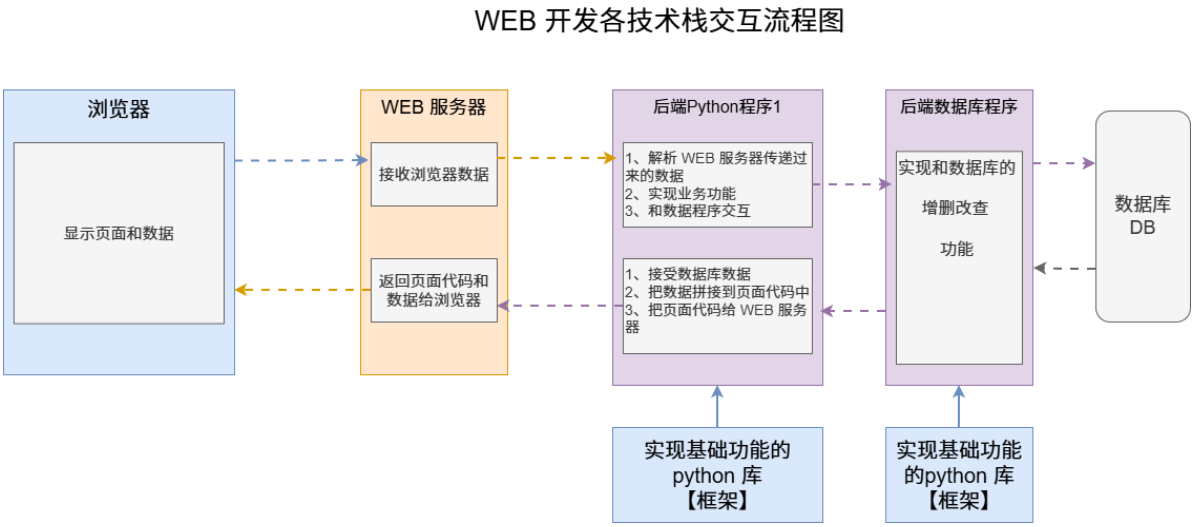
学 PythonWeb 的目标: 在浏览器中展现和操作数据库中的数据。



WEB 开发需要使用到的技术

1. **前端技术**:
   - **基础三件套**：HTML、CSS、JavaScript
   - 模板引擎：可直接在 HTML 中嵌入 Python 变量和逻辑（如循环、条件判断）
2. **WEB 服务器**：提供接收浏览器数据，给浏览器反馈页面数据的功能
3. **核心框架**:
   - 全栈框架：Django：内置浏览器和程序的交互、程序和数据库的交互功能
   - 微框架：Flask：仅仅支持 浏览器和 程序的交互

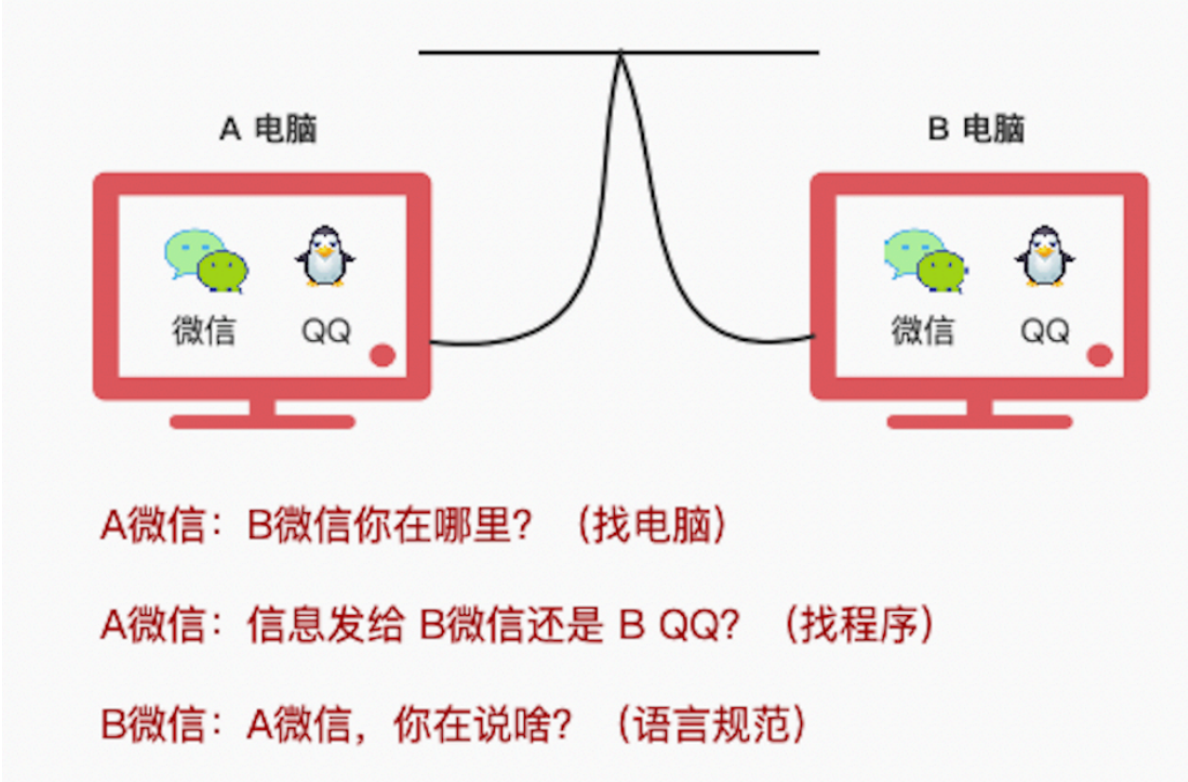
4. **数据库对接**：ORM 工具
   - `Django ORM`：Django 内置（无需额外配置，与框架深度整合）
   - `SQLAlchemy`：独立 ORM 库，支持多种数据库，适合 Flask、Pyramid 等框架。
   - `Peewee` / `Tortoise-ORM`（异步）：轻量级 ORM，适合小型项目。



四、WEB 程序开发

WEB 程序：就是可以基于网络 通过浏览器访问到的程序，可以给浏览器发送 网页的程序，同称WEB 程序。

4.1 浏览器和 WEB 程序交互三大要素



4.1.1 IP 地址

IP 地址：互联网协议地址（Internet Protocol Address）： 在网络中区分一台电脑的唯一标识，好比咱们的身份证号码

IP地址分类

- IPv4：是一个 32 位的二进制数，通常被分为 4 个字节，表示成`a.b.c.d` 的形式，例如 `192.168.65.100` 。其中 a、b、c、d 都是0~255之间的十进制整数，那么最多可以表示大约43亿个。
- IPv6：为了扩大地址空间，通过 IPv6 重新定义地址空间，采用 128位地址长度，每 16 个二进制一组，分成8组十六进制数，表示成`ABCD:EF01:2345:6789:ABCD:EF01:2345:6789`，号称可以为全世界的每一粒沙子编上一个网址，这样就解决了网络地址资源数量不够的问题。

域名: 映射到某个 ip 的一组英文单词组合(baidu.cn)，为了人类更好记住一个网络程序所在的电脑地址.

常用命令

- 查看本机 IP 地址，在**控制台输入**

```
ipconfig
```

- 检查网络是否连通，在控制台输入

```
ping 空格 IP地址/域名
ping www.baidu.com
ping 182.61.200.108
```

特殊的 IP 地址

- 本机 IP 地址：`127.0.0.1`、`localhost`

4.1.2 端口号

浏览器和 WEB 程序通信，本质上是两个进程（应用软件）的通信。每台计算机都有很多的软件，那么在网络通信时，如何区分这些软件呢?

如果说 **IP地址** 可以唯一标识网络中的设备，那么**端口号**就可以唯一标识电脑中的软件（应用程序）。

- **端口号：用两个字节表示的整数，它的取值范围是 0~65535。**其中，0~1023之间的端口号用于一些知名的网络服务和应用，普通的应用程序需要使用 1024 以上的端口号。如果端口号被另外一个服务或应用所占用，会导致当前程序启动失败。

4.1.3 通信协议

通信协议：软件和软件之间交换数据共用的格式，浏览器和 WEB 程序交互采用 HTTP 协议，通过这个协议，确定了双方按哪种格式来传输数据。

4.1.3.1 HTTP 协议的组成

- 请求：浏览器访问服务器（带数据给服务器）的过程
- 响应：服务器反馈数据给浏览器的过程

浏览器访问 WEB 服务器，这个过程称为 请求：request

例： 访问 http://www.baidu.com

WEB 服务器给浏览器反馈数据，这个过程称为响应:response

比如访问百度首页，返回百度首页的 html 等代码给浏览器，

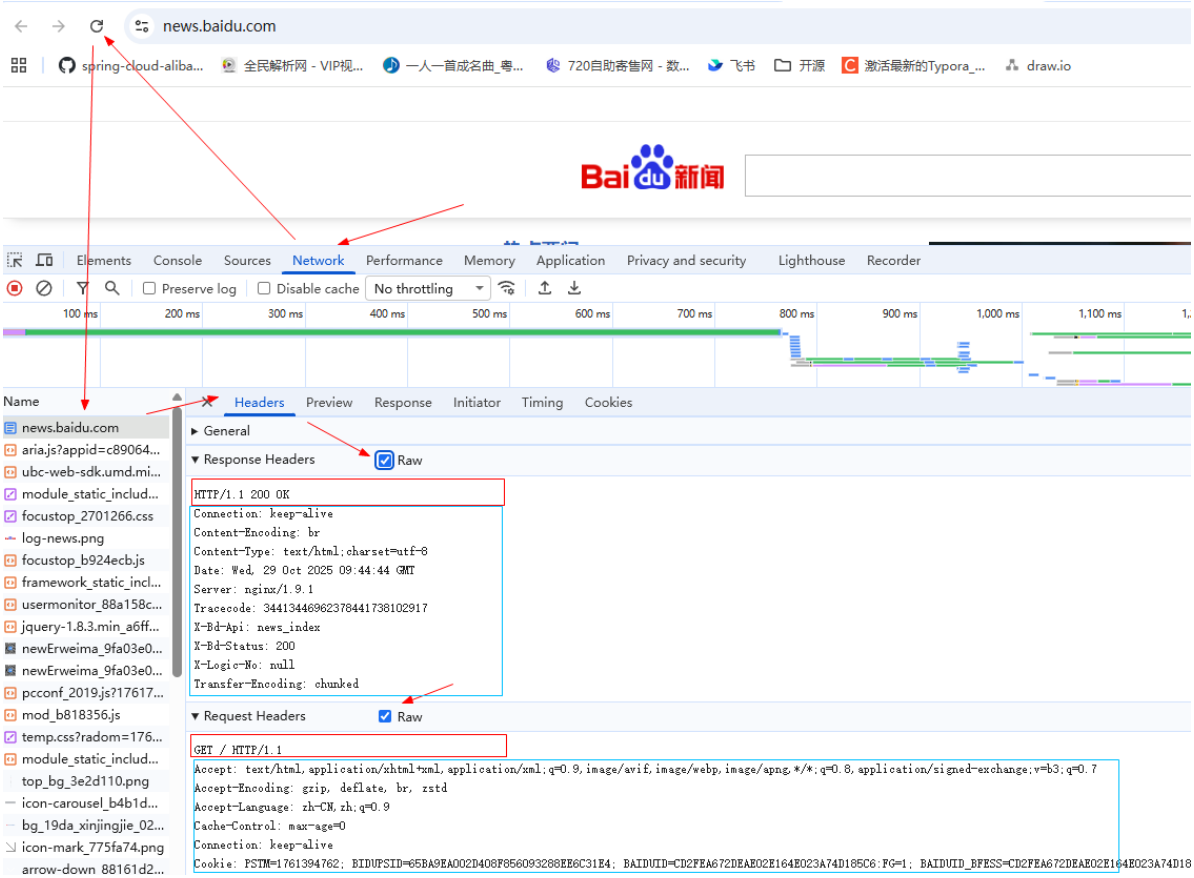
浏览器解析页面代码之后就是我们看到的效果

4.1.3.2 请求和响应数据格式

- **HTTP 请求数据格式**
  - 请求首行：http版本,当前请求的方式
  - 请求头：一大堆 key:value 键值对行 \r\n
  - 请求体： get 没有，post 有
- **HTTP 响应数据格式**
  - 响应首行：(标识 http 版本,响应状态码)
  - 响应头：一大堆 key:value 键值对行 \r\n
  - 响应体：返回给浏览器的页面数据

4.1.3.3 查看请求和响应数据

浏览器 F12 或 （Fn + F12）查看请求和响应数据



4.2 搭建WEB服务器

实现步骤：

```
# 1、构建服务器端
# 2、接收客户端（浏览器）请求的数据
# 3、给客户端（浏览器）反馈数据
    # 3.1、处理响应行和响应头
    # 3.2、处理不同访问路径反馈不同内容问题
# 4、连接MySQL获取数据库数据（连标直取）
```

4.2.1 基础服务器

server.py

```
import socket

# 1、构建服务器端
s = socket.socket()
s.bind(('127.0.0.1',8001))
s.listen(5)

while True:
    conn,addr = s.accept()
    # 2、接收客户端（浏览器）请求的数据
    data = conn.recv(1024)
    print(data)

    # 3、给客户端（浏览器）反馈数据
```

```
    # 3.1、处理响应行和响应头
    #\r\n\r\n 表示响应头结束。此处未提供任何响应头字段（如Content-Type），直接进入响应体。
    conn.send(b"HTTP/1.1 200 OK \r\n")  # 拼接响应的前三行
    conn.send(b'Content-Type:text/html;charset=utf-8 \r\n\r\n\')
    conn.send(b'Hello World!') #将b 传输成二进制传输给浏览器

    # 关闭和浏览器的连接
    conn.close()
```

4.2.2 页面返回服务器

获取数据库数据

user_db.py

```
import pymysql

def get_date():
    c = pymysql.connect(
                host='127.0.0.1',
                port=3306,
                user='root',
                password="admin",
                database='mydb'
                )
    # 返回数据的格式是字典
    cur = c.cursor(pymysql.cursors.DictCursor)
    # 构建sql
    sql = 'select * from user'
    cur.execute(sql)
    data = cur.fetchall()  # 以列表的形式,每一个元素是字典

    return data
```

根据指定连接获取返回不同的数据

server.py

```
import socket
import user_db
# 1、构建服务器端
s = socket.socket()
s.bind(('127.0.0.1', 8001))
s.listen(5)

while True:
    conn, addr = s.accept()
    # 2、接收客户端（浏览器）请求的数据
    data = conn.recv(1024)
    print(data)

    # 3、给客户端（浏览器）反馈数据
```

```
    # 3.1、处理响应行和响应头
    # \r\n\r\n 表示响应头结束。此处未提供任何响应头字段（如Content-Type），直接进入响应体。
    conn.send(b"HTTP/1.1 200 OK \r\n")  # 拼接响应的前三行
    conn.send(b'Content-Type:text/html;charset=utf-8 \r\n\r\n')
    # conn.send(b'Hello World!') #将b 传输成二进制传输给浏览器

    # ----------------------2----------------------------
    # 3.2、处理不同访问路劲反馈不同内容问题
    # 获取到路由
    path = data.decode().split(' ')[1]
    print(path)
    if path == '/test':
        with open('test.html', 'r', encoding='utf-8') as f:
            file_data = f.read()
            conn.send(file_data.encode())
    elif path == '/get_data':
        # ----------------3---------------------
        # 从数据库查询出来
        data = user_db.get_date()
        conn.send(str(data).encode('utf-8'))
        # -------------------------------------
    else:
        conn.send('没有这个页面'.encode('utf-8'))

    # 关闭和浏览器的连接
    conn.close()
```

4.2.3 数据库数据返回

增加数据的返回

```
import socket
import pymysql

s = socket.socket()
#构建服务端
s.bind(('127.0.0.1',8001))
s.listen(5)

def get_date():
    c = pymysql.connect(
    host='127.0.0.1',
    port=3306,
    user='root',
    password="admin",
    database='wydb'
    )
    # 返回数据的格式是字典
    cur = c.cursor(pymysql.cursors.DictCursor)
    # 构建sql
    sql = 'select * from wangyijob'
    cur.execute(sql)
    data = cur.fetchall()  # 以列表的形式,每一个元素是字典

    return data
```

```
while True:
    conn,addr = s.accept()
    data = conn.recv(1024)  #接收客户端的数据
    #\r\n\r\n 表示响应头结束。此处未提供任何响应头字段（如Content-Type），直接进入响应体。
    conn.send(b"HTTP/1.1 200 OK \r\n")  # 拼接响应的前三行
    conn.send(b'Content-Type:text/html;charset=utf-8 \r\n\r\n\')

    #获取到路由
    path = data.decode().split(' ')[1]
    print(path)
    if path =='/test':
        with open('test.html','r',encoding='utf-8') as f:
            file_data = f.read()
        conn.send(file_data.encode())
    elif path =='/get_data':
        #从数据库查询出来
        db_date = str(get_date())
        # 通过html文件的形式发送给客户端
        conn.send(db_date.encode('gbk'))
    else:
        conn.send('没有这个页面'.encode('gbk'))

    conn.close()
    # conn.send(b'Hello World!') #将b 传输成二进制传输给浏览器
```

server.py

```
import socket
import pymysql

s = socket.socket()
#构建服务端
s.bind(('127.0.0.1',8001))
s.listen(5)

def get_date():
    c = pymysql.connect(
    host='127.0.0.1',
    port=3306,
    user='root',
    password="wolfcode",
    database='wydb'
    )
    # 返回数据的格式是字典
    cur = c.cursor(pymysql.cursors.DictCursor)
    # 构建sql
    sql = 'select * from wangyijob'
    cur.execute(sql)
    data = cur.fetchall()  # 以列表的形式,每一个元素是字典

    return data

while True:
    conn,addr = s.accept()
```

```
    data = conn.recv(1024)  #接收客户端的数据
    #\r\n\r\n 表示响应头结束。此处未提供任何响应头字段（如Content-Type），直接进入响应体。
    conn.send(b"HTTP/1.1 200 OK \r\n")  # 拼接响应的前三行
    conn.send(b'Content-Type:text/html;charset=utf-8 \r\n\r\n\')

    #获取到路由
    path = data.decode().split(' ')[1]
    print(path)
    if path =='/test':
        with open('test.html','r',encoding='utf-8') as f:
            file_data = f.read()
        conn.send(file_data.encode())
    elif path =='/get_data':
        #从数据库查询出来
        db_date = str(get_date())
        # 通过html文件的形式发送给客户端
        conn.send(db_date.encode('gbk'))
    else:
        conn.send('没有这个页面'.encode('gbk'))

    conn.close()
    # conn.send(b'Hello World!') #将b 传输成二进制传输给浏览器
```

可以将 web-框架拆成3个部分:

Socket:django 使用的wsgi协议叫做 wsgi

路径和视图函数对应关系(路由匹配)

模版语法:(html页面可以直接使用python的变量)

4.2.4 问题说明

1. 浏览器带过来的数据需要咱们用代码一点一点截取出来，太繁琐
2. 返回的页面和数据都需要咱们自行处理，麻烦
3. 服务器中的路由判断需要咱们自己写，html 多时需要写一堆的判断代码

解决方案:

- 1、2 问题使用别人写好的 WEB 服务器（WSGI 服务器）
- 3 问题使用 WEB 框架解决（Django/Flask：都内置了WSGI 服务器）
- 所以：1.2.3 问题都可以使用 Django/Flask 框架来解决

4.3 框架介绍

框架是一个工具库，已经对基础的代码进行了实现，开发者在使用框架时直接调用写好的工具库从而省去很多代码编写，从而提高开发速度。

框架的工作模式:
开发工程师在框架的基础之上开发部分功能 + 框架自身已经完成的功能 = 完整的产品

例子: 小店卖手抓饼

1. 自给自足
    - 自己和面制作手抓饼的饼、酱、配菜
    - 制作手抓饼
2. 使用框架
    - 购买别人做好的饼（框架）、酱（框架）、配菜（框架）
    - 根据客户喜好 组装手抓饼

4.3.1 Web 框架

Web 框架是用于简化 Web 应用程序开发的工具集，提供标准化的结构和组件，

封装了 HTTP **请求/响应**处理、模板渲染等通用功能。

核心作用

- **简化开发**：提供现成的模块（如路由、模板引擎），减少重复代码。
- **提高安全性**：内置防护机制（如 CSRF、表单数据验证等）。
- **提升效率**：通过约定优于配置（Convention over Configuration）减少决策成本。
- **支持扩展**：通过插件或中间件扩展功能。

4.3.2 常见 Web 框架

框架	语言	类型	核心特性
`Django`	Python	**全栈**	ORM、Admin 后台、内置安全防护
Flask	Python	微框架	轻量级、灵活扩展
SpringMVC	Java	微框架	轻量级、灵活扩展
Express.js	JavaScript	微框架	高性能、适合单页应用（SPA）
Ruby on Rails	Ruby	全栈	约定优于配置、快速开发

咱们这个阶段就在学习 全栈框架 Django

五、Django 全栈框架

5.1 Django 核心优势

1. **全栈能力**：
    - 内置 ORM、模板引擎、用户认证、管理后台。
2. **安全性**：
    - 防止 SQL 注入、XSS 攻击、CSRF 攻击。
3. **可扩展性**：
    - 支持插件（如 `django-rest-framework` 构建 API）。

4. **社区与生态**：
    - 丰富的第三方库（如 `django-allauth` 处理社交登录）。

5.2 Django 典型应用场景

- **内容管理系统（CMS）**：如博客、新闻网站。
- **电商平台**：商品管理、购物车、订单系统。
- **社交网络**：用户注册、动态发布、消息通知。

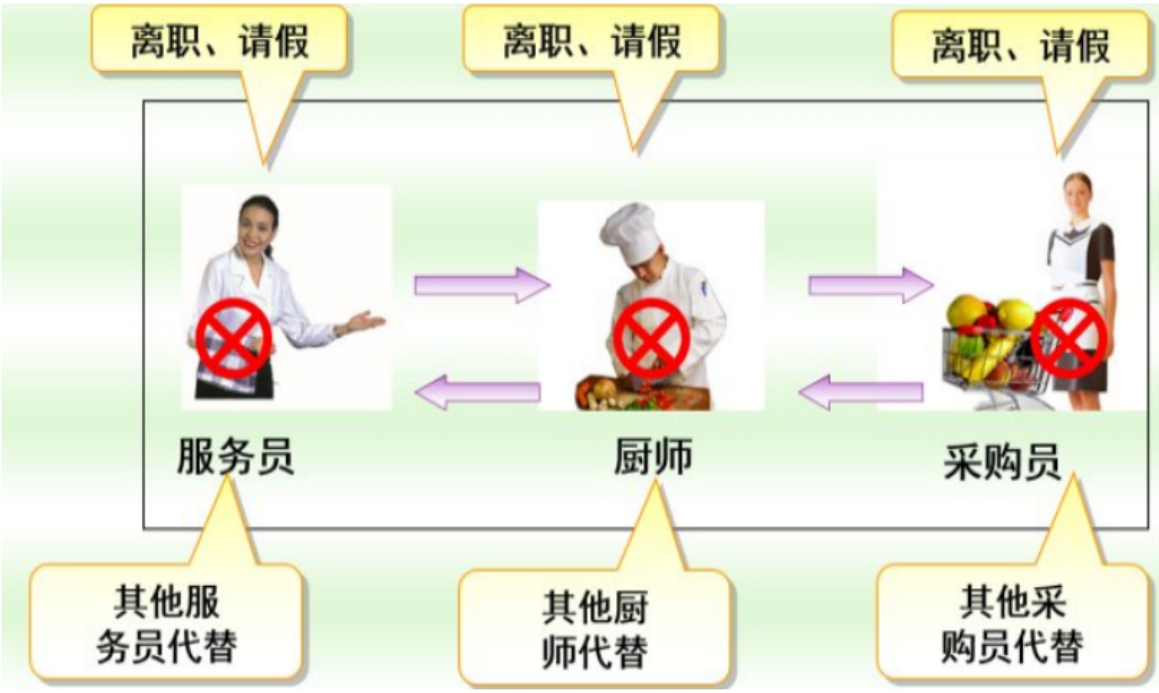
5.3 微框架对比

	全栈框架 (Django、Ruby on Rails)	微框架 (Flask、Express.js)
特点	提供完整的解决方案（路由、模板、ORM、用户认证等） 适合复杂项目，如电商平台、社交网络。	轻量级，仅提供核心功能（路由、请求处理） 需要手动集成数据库、模板引擎等组件。
优点	开箱即用，减少集成第三方库的成本。 社区支持完善，文档丰富。	灵活性高，适合小型项目或 API 开发。

六、项目设计模式

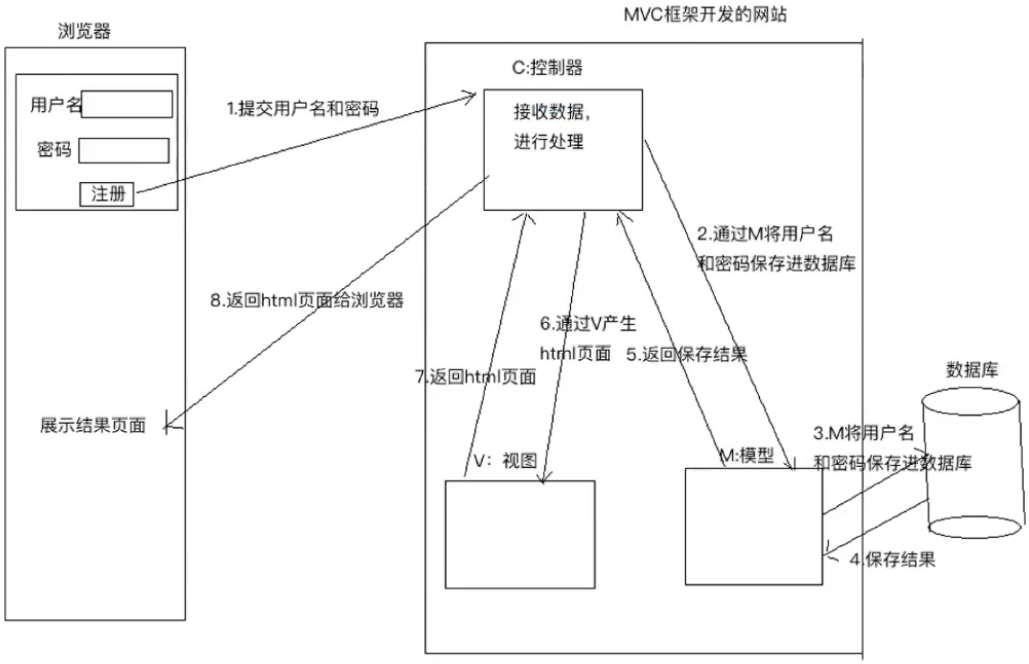
6.1 MVC 设计模式

MVC 是一种软件的设计模式，其产生的的理念就是：`分工，责任分离,解耦`



6.1.1 注册 MVC 模式流程

> 用户请求 → 路由（URL 映射） → 视图（业务逻辑） → 模板（渲染 HTML） → 用户响应



6.1.2 MVC 理解

- M：Model，模型，完成和数据库交互功能。
- V： View，视图，产生 html 页面。
- C：Controller，控制器，接收请求，进行处理，与 M 和 V 进行交互，返回html页面和数据

6.2 MVT 设计模式

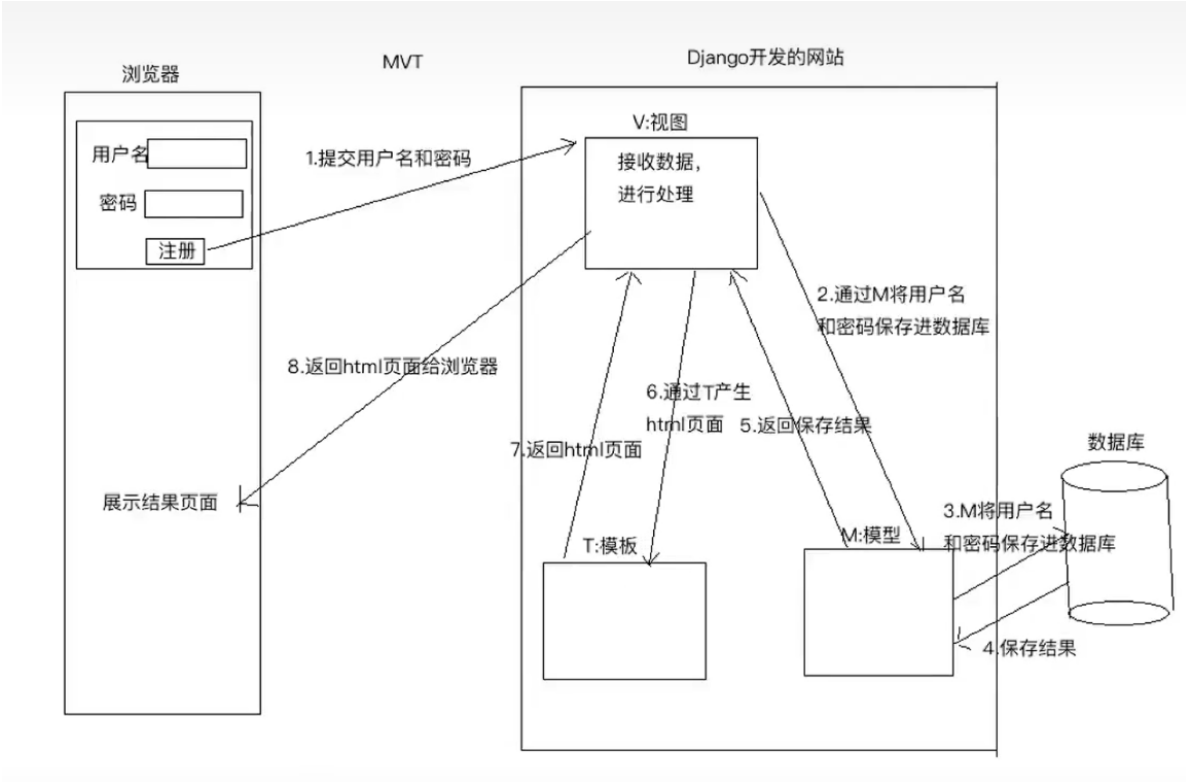
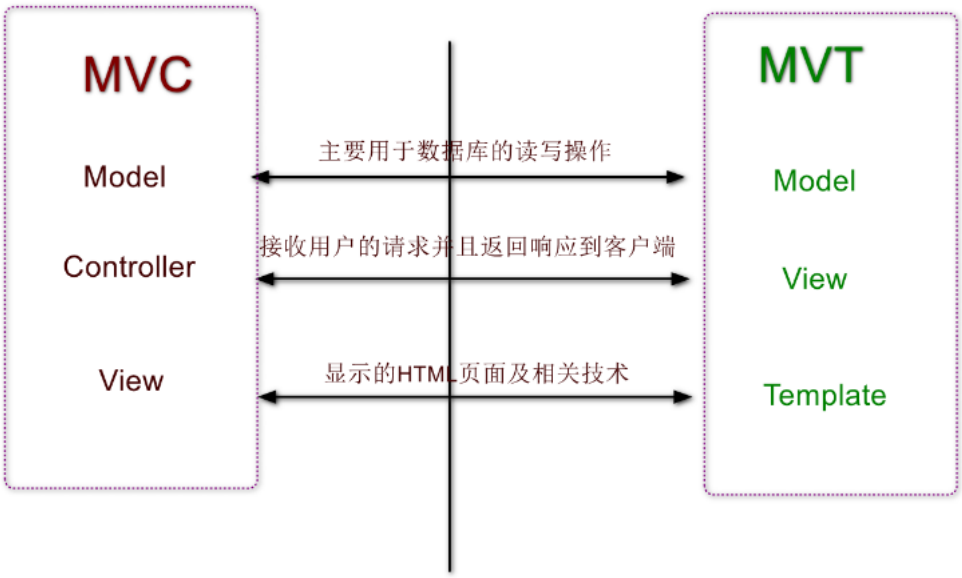
Diango 全栈框架采用的设计模式，遵循**快速开发和 DRY** 原则。

DRY：Do not repeat yourself. 不要自己去重复一些工作。

MVT 含义：

- **M**：Model，模型 和 MVC 中 `M` 功能相同,也是和数据库进行交互
- **V**：View，视图 和 MVC 的 `C` 功能相同,接收请求,进行处理,与M和T进行交互,返回应答
- **T**：Template，模板 和 MVC 中 `V` 功能相同,产生HTML页面.

6.2.1 注册 MVT 模式流程图



6.2.2 MVT--MVC 区别